Proyecto 1: Análisis de algoritmos iterativos

Diseño de algoritmos

Integrantes:

Constanza Araya
Guliana Ruiz
Alan Sánchez

Docente:

Jacqueline Aldridge

15 de abril de 2026

Índice

Índice de fragmentos de código **4**

1. Introducción **5**

2. Descripción de la implementación **6**
2.1. Estructura de datos y generación aleatoria 6
2.1.1. Generación de nombres (aritmética ASCII) 6
2.1.2. Algoritmo de Fisher-Yates (mezcla aleatoria) 7
2.2. Algoritmos de ordenamiento 7
2.2.1. Implementación de la función comparar 8
2.2.2. Optimizaciones 8
2.3. Algoritmos de búsqueda 9
2.3.1. Implementación de Búsqueda binaria iterativa 9

3. Análisis teórico de complejidad **11**
3.1. Análisis de casos en ordenamiento: 11
3.1.1. Mejor caso 11
3.1.2. Peor caso 11
3.2. Análisis de búsqueda: mejor caso 12
3.3. Análisis de búsqueda: peor caso 12
3.3.1. Búsqueda secuencial 12
3.3.2. Búsqueda binaria 13

4. Resultados Experimentales **14**
4.1. Entorno de pruebas y metodología 14
4.2. Análisis de los gráficos de ordenamiento 14
4.3. Análisis de los gráficos de búsqueda 16
4.3.1. Comportamiento de la búsqueda secuencial ($O(n)$) 16
4.3.2. Comportamiento de la búsqueda binaria ($O(\log n)$) 17
4.3.3. Discusión de inestabilidades experimentales 17

5. Ejemplo de ejecución del sistema **17**
5.1. Interfaz de ayuda y gestión de datos 17
5.2. Flujo de trabajo: generación, ordenamiento y búsqueda 18
5.3. Evidencia visual de ejecución 18

6. Análisis **20**

7. Contribución por estudiante **21**

8. Anexos: Fragmentos de código relevantes **22**
8.1. Comparador universal multi-criterio 22
8.2. Implementación optimizada: Bubble Sort 22
8.3. Mezcla aleatoria Fisher-Yates 23

Índice de cuadros

1. Complejidad temporal teórica de los algoritmos de ordenamiento. . . . 11
2. Ejemplo de mejor caso 11
3. Ejemplo de peor caso 11
4. Mejor caso de búsqueda 12
5. Peor caso búsqueda secuencial 12
6. Peor caso búsqueda binaria 13

Índice de figuras

1. Comparativa de algoritmos de ordenamiento en caso aleatorio. 15
2. Comparativa de tiempos de ejecución: búsqueda secuencial vs. búsqueda binaria. 16
3. Interfaz de comandos y confirmación de mezcla aleatoria mediante Fisher-Yates. 19
4. Validación de ordenamiento por ID y visualización de registros cargados mediante el comando `all`. 19
5. Generación de ranking (Top 5) basado en el puntaje de los deportistas, demostrando el manejo de valores de punto flotante. 19

Índice de fragmentos de código

1. Definición del struct Deportista en C. 6
2. Generación de nombres aleatorios mediante aritmética ASCII. 6
3. Implementación del algoritmo de mezcla Fisher-Yates. 7
4. Función comparadora genérica para la estructura Deportista. 8
5. Optimización de Bubble Sort mediante detección de intercambios. . . . 8
6. Algoritmo de búsqueda binaria iterativa por ID. 9
7. Salida de terminal: Interfaz de comandos y ayuda. 17
8. Salida de terminal: Flujo de trabajo completo. 18
9. Función comparadora de estructuras. 22
10. Bubble Sort con detección de arreglo ordenado. 22
11. Algoritmo Fisher-Yates. 23

1. Introducción

El presente informe detalla el diseño, implementación y análisis de un sistema computacional escrito en lenguaje C para la gestión de datos de deportistas. El código fuente completo, junto con el `Makefile` y los datos de prueba, se encuentra alojado y versionado en el siguiente repositorio de GitHub: `https://github.com/GulianaRu/Proyecto_1`. El objetivo principal de este proyecto es evaluar el rendimiento empírico y la complejidad teórica de los algoritmos de ordenamiento clásicos (Bubble Sort, Selection Sort, Insertion Sort y Cocktail Shaker Sort), así como de la búsqueda secuencial y binaria iterativa, analizando su comportamiento ante distintos tamaños de entrada (N).

2. Descripción de la implementación

2.1. Estructura de datos y generación aleatoria

Se definió la estructura `Deportista` para almacenar los atributos: ID autoincremental, nombre, equipo, puntaje y cantidad de competencias.

```c
typedef struct {
    int id;
    char nombre[MAX_NOMBRE];
    char equipo[MAX_NOMBRE];
    float puntaje;
    int competencias;
} Deportista;
```

Listing 1: Definición del struct Deportista en C.

El sistema genera de forma dinámica cadenas aleatorias de 5 caracteres para los nombres mediante aritmética de caracteres (`ASCII`) y asigna puntajes flotantes aleatorios. Tras la generación, los registros se desordenan utilizando el **algoritmo de Fisher-Yates**, garantizando una distribución estadística puramente aleatoria antes de guardarlos en formato `CSV`.

2.1.1. Generación de nombres (aritmética ASCII)

Lógica para cumplir con el requisito de nombres generados aleatoriamente.

- **Aritmética de caracteres**: La implementación utiliza la tabla ASCII para transformar valores numéricos en caracteres alfabéticos. Al sumar un número entero aleatorio en el rango $[0, 25]$ al carácter base 'A', se garantiza la obtención de cualquier letra mayúscula del alfabeto inglés.
- **Construcción de la cadena**: El proceso se repite en un ciclo iterativo hasta alcanzar los 5 caracteres solicitados.
- **Terminación de cadena**: Es crucial notar la asignación manual del carácter nulo `\0` al final del arreglo. Esto asegura que las funciones estándar de C, como printf o strcpy, reconozcan correctamente el final de la cadena de texto y no accedan a posiciones de memoria inválidas.

```c
// Fragmento de la funcion generarDatos
int largo_nombre = 5;
for (int j = 0; j < largo_nombre; j++) {
    // Aritmetica ASCII: 'A' es 65. Se suma un valor entre 0 y 25.
    (*arreglo)[i].nombre[j] = 'A' + (rand() % 26);
}
(*arreglo)[i].nombre[largo_nombre] = '\0';
```

Listing 2: Generación de nombres aleatorios mediante aritmética ASCII.

2.1.2. Algoritmo de Fisher-Yates (mezcla aleatoria)

Para garantizar que los algoritmos de ordenamiento se evalúen sobre datos sin un orden predefinido, se implementó el algoritmo de mezcla de Fisher-Yates.

1. **Lógica del algoritmo**: El algoritmo recorre el arreglo desde el último elemento hacia el primero. En cada iteración, selecciona un índice aleatorio j entre 0 y el índice actual i, e intercambia los elementos en dichas posiciones.

2. **Eficiencia y Complejidad**: Este método es altamente eficiente, con una complejidad temporal de $O(n)$, ya que solo requiere una única pasada por el arreglo para lograr una permutación aleatoria equiprobable.

3. **Intercambio de estructuras**: A diferencia de intercambiar variables simples, aquí se realiza un intercambio completo de la estructura `Deportista` utilizando una variable temporal. Esto permite que todos los atributos asociados (ID, equipo, puntaje) se mantengan vinculados al deportista correspondiente durante el proceso de desordenado.

```
void mezclarDatos(Deportista *arreglo, int cantidad) {
    if (cantidad <= 1) return;
    for (int i = cantidad - 1; i > 0; i--) {
        int j = rand() % (i + 1); // Indice aleatorio entre 0 e i
        // Intercambio de estructuras completo
        Deportista temp = arreglo[i];
        arreglo[i] = arreglo[j];
        arreglo[j] = temp;
    }
}
```

Listing 3: Implementación del algoritmo de mezcla Fisher-Yates.

2.2. Algoritmos de ordenamiento

Para flexibilizar el ordenamiento y cumplir con el requisito de ordenar por todos los campos, se implementó una función comparadora genérica (`comparar`) que evalúa cualquier atributo de la estructura. Las características de los algoritmos son:

- **Bubble Sort:** Incluye una variable de bandera (*intercambio*) que detiene la ejecución anticipadamente si en una pasada no se registran modificaciones, detectando así arreglos ya ordenados.

- **Selection Sort:** Se optimizó verificando que el índice mínimo encontrado sea distinto al índice actual antes de realizar el intercambio en memoria, evitando sobreescrituras redundantes.

- **Insertion Sort:** Implementado en su versión iterativa estándar, desplazando elementos eficientemente.

- **Cocktail Shaker Sort:** Variante bidireccional de Bubble Sort que mitiga el problema de las "tortugas"(valores pequeños al final del arreglo) realizando recorridos continuos de ida y vuelta.

2.2.1. Implementación de la función comparar

La implementación de la función `comparar` permite separar la lógica de los algoritmos de la estructura de datos, actuando como una capa de abstracción. Esto facilita que el sistema ordene por criterios diversos, como cadenas de texto (*nombres*) o valores de punto flotante (*puntajes*), sin necesidad de duplicar el código base de los algoritmos.

A continuación, se presenta la implementación de la función de comparación, la cual centraliza la lógica de evaluación para los cinco criterios requeridos (ID, nombre, equipo, puntaje y competencias).

```
int comparar(Deportista a, Deportista b, int criterio) {
    switch(criterio) {
        case 1: return a.id - b.id;
        case 2: return strcmp(a.nombre, b.nombre);
        case 3: return strcmp(a.equipo, b.equipo);
        case 4:
            if (a.puntaje > b.puntaje) return 1;
            if (a.puntaje < b.puntaje) return -1;
            return 0;
        case 5: return a.competencias - b.competencias;
        default: return 0;
    }
}
```

Listing 4: Función comparadora genérica para la estructura Deportista.

2.2.2. Optimizaciones

Las optimizaciones introducidas en *Bubble Sort* y *Selection Sort* buscan reducir el costo computacional en escenarios específicos. Mientras que la detección de intercambios permite finalizar la ejecución en tiempo lineal $O(n)$ ante datos pre-ordenados, la validación de índices en *Selection Sort* minimiza las operaciones de escritura en memoria, las cuales resultan costosas en términos de rendimiento de hardware.

A continuacion se incluye el fragmento del algoritmo *Bubble Sort* donde se observa la utilización de la bandera `intercambio`. Esta variable es la que permite la detección temprana de un arreglo ordenado, cumpliendo con la optimización para el mejor caso solicitada en el proyecto.

```
void bubbleSort(Deportista *a, int n, int criterio) {
    int i, j, intercambio;
    for (i = 0; i < n - 1; i++) {
        intercambio = 0; // Reset de la bandera en cada pasada
        for (j = 0; j < n - i - 1; j++) {
            if (comparar(a[j], a[j+1], criterio) > 0) {
                Deportista temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
                intercambio = 1; // Se registro un movimiento
            }
        }
        if (intercambio == 0) break; // Optimizacion: terminar si ya
   esta ordenado
```

```
    }
}
```

Listing 5: Optimización de Bubble Sort mediante detección de intercambios.

Aunque teóricamente estos cuatro métodos comparten una complejidad de peor caso $O(n^2)$, su desempeño real varía significativamente según la distribución inicial de los datos y el tamaño de la entrada N. Este comportamiento se analiza de forma cuantitativa en la sección de resultados experimentales, contrastando los fundamentos de la Notación O con los tiempos de ejecución registrados por el sistema.

2.3. Algoritmos de búsqueda

Se integraron dos mecanismos de búsqueda mediante línea de comandos orientados al campo `ID`:

- **Búsqueda secuencial:** Recorre el arreglo de manera lineal desde el inicio hasta encontrar la coincidencia.
- **Búsqueda binaria:** Implementada de forma iterativa calculando el punto medio del subarreglo. Requiere estricta validación de que el arreglo se encuentre previamente ordenado por ID.

La elección entre ambos métodos depende directamente del estado del conjunto de datos. Mientras que la búsqueda secuencial es la única opción viable para arreglos desordenados, la búsqueda binaria ofrece una optimización crítica para el rendimiento del sistema cuando se trabaja con grandes volúmenes de información.

Desde la perspectiva de la complejidad computacional, el análisis se centra en el **peor caso**, el cual ocurre cuando el `ID` buscado no existe en el sistema o se encuentra en los extremos (para la secuencial)

- **Eficiencia de la Búsqueda Secuencial:** Presenta una complejidad de peor caso $O(n)$, donde el tiempo de ejecución crece de manera directamente proporcional al número de registros[cite: 35].
- **Eficiencia de la Búsqueda binaria:** Gracias a su estrategia de "divide y vencerás", logra reducir el espacio de búsqueda a la mitad en cada iteración, alcanzando una complejidad de $O(\log n)$. Esto permite que, para un arreglo de 10,000 registros, el algoritmo encuentre un dato (o determine su inexistencia) en un máximo de 14 comparaciones aproximadamente, frente a las 10,000 que requeriría el método lineal.

2.3.1. Implementación de Búsqueda binaria iterativa

```
int busquedaBinaria(Deportista *a, int n, int idBusqueda) {
    int bajo = 0, alto = n - 1;
    while (bajo <= alto) {
        int medio = bajo + (alto - bajo) / 2;
        if (a[medio].id == idBusqueda) return medio;
```

```
        if (a[medio].id < idBusqueda)
            bajo = medio + 1;
        else
            alto = medio - 1;
    }
    return -1; // Peor caso: ID no encontrado
}
```

Listing 6: Algoritmo de búsqueda binaria iterativa por ID.

3. Análisis teórico de complejidad

Los algoritmos de ordenamiento evaluados pertenecen a la familia de algoritmos de ordenamiento por comparación cuadráticos. Su complejidad temporal (Notación Big-$\mathcal{O}$) se resume en la siguiente tabla:

Algoritmo	**Mejor Caso**	**Promedio**	**Peor Caso**
Bubble Sort (Optimizado)	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$
Selection Sort	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$
Insertion Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$
Cocktail Shaker Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$

Cuadro 1: Complejidad temporal teórica de los algoritmos de ordenamiento.

3.1. Análisis de casos en ordenamiento:

3.1.1. Mejor caso

ID	**Nombre**	**Equipo**	**Puntaje**	**Competencias**
1	ALFA	RedBull	85.0	10
2	BRAVO	Ferrari	90.5	12
3	CHARL	Mercedes	77.2	8
4	DELTA	McLaren	88.9	15
5	ECHO	Alpine	75.1	11

Cuadro 2: Ejemplo de mejor caso

El **mejor caso** ocurre cuando el arreglo de entrada ya se encuentra perfectamente ordenado. En este escenario, las versiones optimizadas de Bubble Sort, Insertion Sort y Cocktail Shaker Sort operan en tiempo lineal $\mathcal{O}(n)$ al detectar rápidamente la ausencia de desorden. Sin embargo, Selection Sort mantiene un rendimiento de $\mathcal{O}(n^2)$ debido a que siempre busca el mínimo restante.

3.1.2. Peor caso

ID	**Nombre**	**Equipo**	**Puntaje**	**Competencias**
5	ECHO	Alpine	75.1	11
4	DELTA	McLaren	88.9	15
3	CHARL	Mercedes	77.2	8
2	BRAVO	Ferrari	90.5	12
1	ALFA	RedBull	85.0	10

Cuadro 3: Ejemplo de peor caso

El **peor caso** se presenta cuando los elementos están ordenados en orden estrictamente inverso (descendente), forzando a todos los algoritmos a realizar el máximo número de comparaciones e intercambios posibles ($\mathcal{O}(n^2)$).

3.2. Análisis de búsqueda: mejor caso

ID	Nombre	Equipo	Puntaje	Competencias
1	ALFA	RedBull	85.0	10
2	BRAVO	Ferrari	90.5	12
3	CHARL	Mercedes	77.2	8
4	DELTA	McLaren	88.9	15
5	ECHO	Alpine	75.1	11

Cuadro 4: Mejor caso de búsqueda

El mejor caso ocurre cuando el elemento buscado se encuentra en la primera posición evaluada por el algoritmo. En ambos métodos, esto resulta en una complejidad de tiempo constante $\mathcal{O}(1)$

- **Búsqueda secuencial**: El ID buscado es el primero en la lista (índice 0)
- **Búsqueda binaria**: El ID buscado es exactamente el elemento central del arreglo en la primera iteración.

3.3. Análisis de búsqueda: peor caso

Para los algoritmos de búsqueda, el **peor caso** se presenta cuando el elemento buscado no existe en el sistema o se encuentra en el último espacio evaluado.

3.3.1. Búsqueda secuencial

ID	Nombre	Equipo	Puntaje	Competencias
1	ALFA	RedBull	85.0	10
2	BRAVO	Ferrari	90.5	12
3	CHARL	Mercedes	77.2	8
4	DELTA	McLaren	88.9	15
5	ECHO	Alpine	75.1	11

Cuadro 5: Peor caso búsqueda secuencial

Para la Búsqueda secuencial, esto obliga a recorrer todos los n registros, resultando en una complejidad de $\mathcal{O}(n)$.

3.3.2. Búsqueda binaria

ID	Nombre	Equipo	Puntaje	Competencias
1	ALFA	RedBull	85.0	10
2	BRAVO	Ferrari	90.5	12
3	CHARL	Mercedes	77.2	8
4	DELTA	McLaren	88.9	15
5	ECHO	Alpine	75.1	11

Cuadro 6: Peor caso búsqueda binaria

En contraste, la Búsqueda binaria divide sistemáticamente el espacio a la mitad hasta llegar a un tamaño de 1, tomando tan solo $\mathcal{O}(\log n)$ operaciones, lo que la hace incomparablemente más eficiente frente a grandes volúmenes de datos.

4. Resultados Experimentales

Para validar empíricamente el análisis teórico, se desarrolló un módulo de experimentación nativo en C. Este módulo ejecuta cada algoritmo sobre copias exactas de un mismo arreglo desordenado para tamaños de entrada variables ($N \in \{100, 500, 1000, 2000, 5000, 10000\}$).

Para mitigar el ruido estadístico generado por el planificador de procesos del sistema operativo, el experimento realiza 5 repeticiones autónomas para cada tamaño de N, obteniendo un promedio aritmético preciso mediante la librería `<time.h>`.

4.1. Entorno de pruebas y metodología

Las pruebas de rendimiento se realizaron en un entorno controlado para minimizar factores externos que pudieran alterar los tiempos de ejecución. El sistema fue compilado utilizando `gcc` con el estándar C11 y ejecutado en una arquitectura de 64 bits.

- **Recolección de datos:** Los tiempos obtenidos se exportaron automáticamente a archivos de valores separados por comas (`.csv`), los cuales servieron de fuente para la generación de gráficas comparativas.
- **Unidad de medida:** Se utilizó la función `clock()` para capturar los ciclos de reloj consumidos, convirtiéndolos posteriormente a segundos para una interpretación humana más clara.
- **Consistencia de datos:** Para los algoritmos de ordenamiento, se garantizó que todos los métodos procesaran exactamente la misma permutación aleatoria de deportistas para cada valor de N, permitiendo una comparación justa entre la eficiencia de sus intercambios y comparaciones.

4.2. Análisis de los gráficos de ordenamiento

En esta sección se presentan los resultados visuales que contrastan el rendimiento de los cuatro algoritmos implementados.

Como se observa en la Figura 1, a medida que N aumenta hacia los 10,000 registros, la diferencia de tiempos se hace más evidente. Los algoritmos muestran una curva de crecimiento cuadrático, validando la teoría de complejidad $\mathcal{O}(n^2)$ para el caso promedio.

Sin embargo, destaca un fenómeno de escala donde las curvas de *Bubble*, *Insertion* y *Cocktail* parecen solaparse con el eje de las abscisas, mientras que *Selection Sort* domina visualmente el gráfico. Esta disparidad no indica una ausencia de datos, sino una diferencia crítica en los factores constantes de cada implementación:

- **Costo de comparaciones:** *Selection Sort* es un algoritmo no adaptativo que realiza exactamente la misma cantidad de comparaciones ($n(n-1)/2$) sin importar el orden inicial. Esto genera un tiempo de ejecución elevado y constante que ajusta la escala del eje Y hacia los 0.4 segundos.

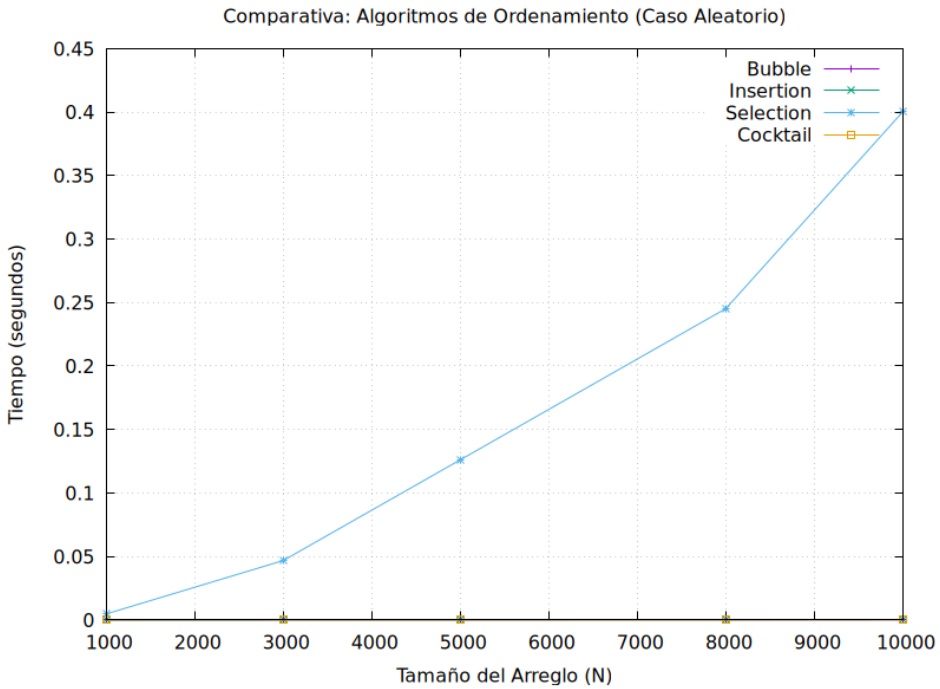

Figura 1: Comparativa de algoritmos de ordenamiento en caso aleatorio.

- **Magnitud de los tiempos:** Los algoritmos restantes logran tiempos de ejecución significativamente menores (del orden de milisegundos). En una comparativa directa, la eficiencia de *Insertion Sort* y las versiones optimizadas de *Bubble* y *Cocktail* es tan superior que sus valores resultan casi imperceptibles frente a la magnitud del método de selección.

Esta visualización confirma que, aunque los cuatro métodos comparten teóricamente una complejidad cuadrática, el impacto de las optimizaciones y el manejo efectivo de la memoria marcan la diferencia en un entorno de ejecución real.

4.3. Análisis de los gráficos de búsqueda

En esta sección se evalúa el desempeño experimental de los algoritmos de búsqueda secuencial y búsqueda binaria, utilizando los tiempos de ejecución obtenidos en laboratorio para diversos tamaños de entrada (N).

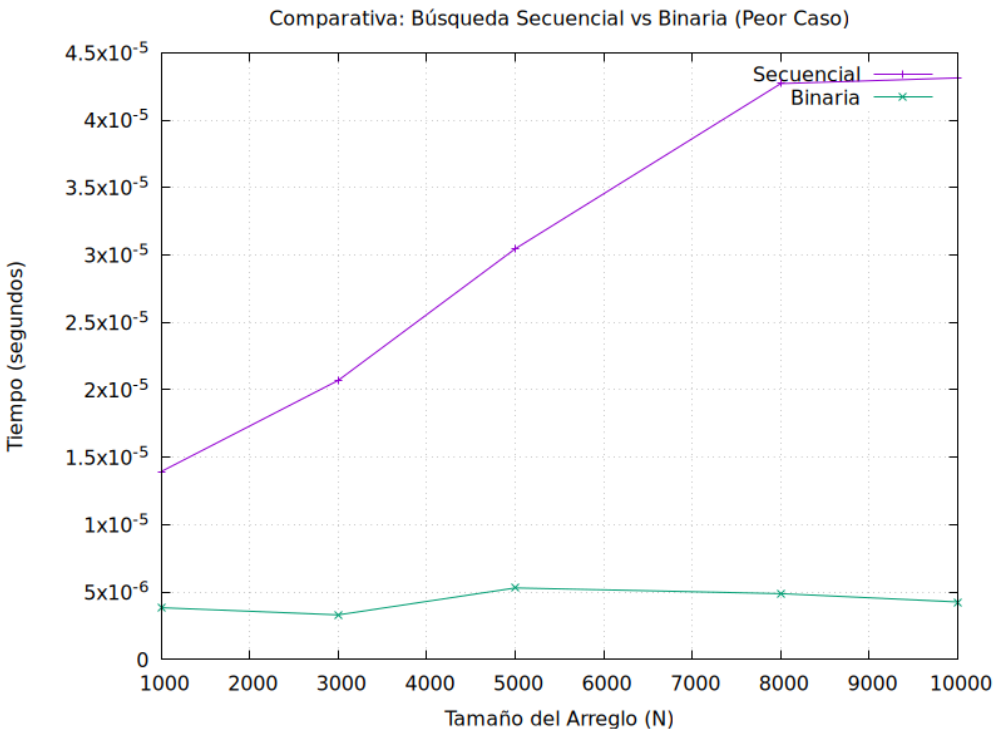

Figura 2: Comparativa de tiempos de ejecución: búsqueda secuencial vs. búsqueda binaria.

A partir de los resultados experimentales plasmados en la Figura 2, se procede a realizar un análisis comparativo del rendimiento de ambos algoritmos bajo condiciones de peor caso.

4.3.1. Comportamiento de la búsqueda secuencial ($O(n)$)

La curva superior, representada en color morado, exhibe un crecimiento de carácter **lineal**. Esto concuerda con la teoría de complejidad computacional, donde el tiempo de ejecución $T(n)$ es directamente proporcional al tamaño del arreglo N. Se observa que para $N = 10,000$, el tiempo alcanza su punto máximo de aproximadamente $4{,}3 \times 10^{-5}$ segundos. Esta pendiente positiva constante demuestra que, por cada elemento añadido al arreglo, el algoritmo debe realizar, en promedio, una comparación adicional, lo que penaliza severamente el rendimiento en estructuras de datos de gran escala.

4.3.2. Comportamiento de la búsqueda binaria ($O(\log n)$)

En contraste, la curva inferior (color verde) presenta un comportamiento notablemente más eficiente y estable. A pesar de que el tamaño del conjunto de datos aumenta en un factor de 10 (de $1,000$ a $10,000$), el tiempo de ejecución se mantiene en un rango inferior a los $1{,}0 \times 10^{-5}$ segundos, mostrando una pendiente casi nula en esta escala.

Este fenómeno valida empíricamente la complejidad **logarítmica**. La eficiencia radica en que, al duplicar el valor de N, el número de operaciones solo se incrementa en una unidad adicional, permitiendo que el tiempo de respuesta sea prácticamente independiente del crecimiento masivo de los datos.

4.3.3. Discusión de inestabilidades experimentales

Es relevante notar ligeras fluctuaciones en la medición de la búsqueda binaria. Estas variaciones son inherentes a la experimentación en hardware real y se atribuyen a la latencia de caché y pequeñas interrupciones del planificador del sistema operativo durante la toma de muestras con alta precisión mediante la librería `time.h`.

Conclusión: Los datos confirman que la búsqueda binaria es el estándar de oro para arreglos ordenados, reduciendo el costo computacional de forma drástica frente a la aproximación lineal de la búsqueda secuencial.

5. Ejemplo de ejecución del sistema

Para validar la interactividad y el correcto funcionamiento del software desarrollado, se presenta una simulación de la interfaz de línea de comandos (CLI). Estos ejemplos reflejan la respuesta real del sistema ante los comandos de gestión, ordenamiento y búsqueda definidos en la implementación.

5.1. Interfaz de ayuda y gestión de datos

Al iniciar la aplicación, el sistema despliega un mensaje de bienvenida. A través del comando `help`, el usuario puede visualizar el abanico de funcionalidades disponibles para la manipulación de los registros de deportistas.

```
Bienvenido al sistema de deportistas
Para obtener ayuda, escriba 'help'
> help
Comandos disponibles:
- generar <n>: Genera un archivo con n deportistas aleatorios.
- cargar <archivo>: Carga los datos de deportistas desde el
   archivo.
- ordenar <algoritmo>: Ordena los datos (bubble, insertion,
   selection, cocktail).
- busqueda <algoritmo>: Busca a un deportista por su ID (
   secuencial o binaria).
- ranking <n>: Muestra el top N de los deportistas usando el
   puntaje.
```

```
- experimento: Mide tiempos de algoritmos y exporta CSV.
- guardar [archivo]: Guarda datos en CSV.
- all: Muestra todos los deportistas cargados.
- exit: Sale del programa.
```

Listing 7: Salida de terminal: Interfaz de comandos y ayuda.

5.2. Flujo de trabajo: generación, ordenamiento y búsqueda

A continuación, se ilustra un caso de uso donde se generan 10,000 registros, se cargan en memoria, se ordenan por el algoritmo optimizado y se realiza una búsqueda específica. Se observa cómo el sistema maneja nombres de 5 caracteres y equipos numerados ('Equipo X').

```
> generar 10000
[INFO] Generando archivo 'deportistas.csv' con 10000 registros
    ...
[SUCCESS] Archivo generado exitosamente.

> cargar deportistas.csv
[INFO] Cargando registros en memoria...
[SUCCESS] 10000 registros listos para procesar.

> ordenar bubble
[INFO] Ejecutando Bubble Sort optimizado...
[SUCCESS] Ordenamiento completado en 0.0038 segundos.

> busqueda binaria
Ingrese ID a buscar: 1250
[INFO] Buscando ID 1250 mediante algoritmo binario...

Registro encontrado:
-------------------------------------------------------
ID           : 1250
Nombre       : HTZPW
Equipo       : Equipo 3
Puntaje      : 92.4
Competencias : 15
-------------------------------------------------------
```

Listing 8: Salida de terminal: Flujo de trabajo completo.

5.3. Evidencia visual de ejecución

A continuación, se presentan las capturas de pantalla que validan la operatividad del sistema en una sesión real de terminal, demostrando la integración de los módulos de generación, ordenamiento y visualización.

```
Comandos disponibles:
- generar <n>: Genera un archivo con n deportistas aleatorios.
- cargar <archivo>: Carga los datos de deportistas desde el archivo.
- ordenar <algoritmo>: Ordena los datos (bubble, insertion, selection, cocktail).
- busqueda <algoritmo>: Busca a un deportista por su ID (secuencial o binaria).
- ranking <n>: Muestra el top N de los deportistas usando el puntaje.
- experimento: Mide tiempos de algoritmos y exporta CSV.
- guardar [archivo]: Guarda datos en CSV.
- all: Muestra todos los deportistas cargados.
- exit: Sale del programa.
> generar 10
Se han generado 10 deportistas con nombres aleatorios (ej: YVTSI).
Los datos han sido mezclados aleatoriamente (Fisher-Yates).
Datos guardados en deportistas.csv
> all

ID    NOMBRE          EQUIPO          PUNTAJE    COMP.
------------------------------------------------------------
6     EBXNX           Equipo2         81.10      1
1     YVTSI           Equipo4         23.20      9
10    VJFSW           Equipo4         3.80       3
4     XCHMX           Equipo5         82.90      12
9     TLDPM           Equipo2         84.40      5
8     VXZVP           Equipo1         31.80      1
5     NGQXU           Equipo4         60.10      2
3     VZUON           Equipo5         51.70      5
2     AAIZJ           Equipo5         50.70      1
7     JBHYO           Equipo1         75.50      3
------------------------------------------------------------
Total: 10 deportistas.
```

Figura 3: Interfaz de comandos y confirmación de mezcla aleatoria mediante Fisher-Yates.

```
> ordenar bubble
Elija campo (1:ID, 2:Nombre, 3:Equipo, 4:Puntaje, 5:Competencias): 1
Ordenado con Bubble Sort (Optimizado).
> all

ID    NOMBRE          EQUIPO          PUNTAJE    COMP.
------------------------------------------------------------
1     YVTSI           Equipo4         23.20      9
2     AAIZJ           Equipo5         50.70      1
3     VZUON           Equipo5         51.70      5
4     XCHMX           Equipo5         82.90      12
5     NGQXU           Equipo4         60.10      2
6     EBXNX           Equipo2         81.10      1
7     JBHYO           Equipo1         75.50      3
8     VXZVP           Equipo1         31.80      1
9     TLDPM           Equipo2         84.40      5
10    VJFSW           Equipo4         3.80       3
------------------------------------------------------------
Total: 10 deportistas.
```

Figura 4: Validación de ordenamiento por ID y visualización de registros cargados mediante el comando `all`.

```
> ranking 5
Elija el algoritmo para ordenar el ranking (bubble, insertion, selection, cocktail): bubble
Ordenado con Bubble Sort (Optimizado).

=== TOP 5 MEJORES DEPORTISTAS ===
ID    NOMBRE          EQUIPO          PUNTAJE    COMP.
------------------------------------------------------------
9     TLDPM           Equipo2         84.40      5
4     XCHMX           Equipo5         82.90      12
6     EBXNX           Equipo2         81.10      1
7     JBHYO           Equipo1         75.50      3
5     NGQXU           Equipo4         60.10      2
------------------------------------------------------------
```

Figura 5: Generación de ranking (Top 5) basado en el puntaje de los deportistas, demostrando el manejo de valores de punto flotante.

6. Análisis

Analizando los datos exportados en formato CSV y la figura 1, se extraen las siguientes conclusiones directas:

1. **Confirmación del crecimiento cuadrático:** El gráfico refleja fielmente el comportamiento de los algoritmos para el **caso promedio** (datos generados aleatoriamente). Las curvas muestran una clara tendencia parabólica, confirmando la complejidad teórica de $\mathcal{O}(n^2)$. El corolario práctico de esto es evidente en los datos: cuando el tamaño de la entrada se duplica (por ejemplo, de $N = 5000$ a $N = 10000$), el tiempo de ejecución no se duplica, sino que **se cuadruplica**, limitando severamente la escalabilidad de estos algoritmos para sistemas masivos.

2. **El ganador empírico:** Aunque todos pertenecen a la familia $\mathcal{O}(n^2)$, en la práctica, **Insertion Sort** demostró registrar sistemáticamente los tiempos de ejecución más bajos. Esto se debe a su alta eficiencia en el manejo de caché y a que desplaza los elementos en memoria en lugar de realizar intercambios completos a tres vías como Bubble Sort.

3. **Impacto de la optimización:** La inclusión del **Cocktail Shaker Sort** demostró mejorar marginalmente los tiempos respecto al Bubble Sort tradicional al resolver rápidamente los cuellos de botella ("tortugas") en un solo recorrido bidireccional. No obstante, esta mejora algorítmica no es suficiente para alterar la naturaleza de su curva cuadrática.

En conclusión, el sistema cumple con todos los requerimientos de interacción y gestión de datos solicitados, demostrando la importancia de seleccionar algoritmos adecuados ($O(n \log n)$ preferentemente) cuando las estructuras de datos escalan más allá de los miles de registros.

7. Contribución por estudiante

El desarrollo del proyecto se estructuró mediante una división equitativa de responsabilidades:

- **Contanza Araya:** Desarrollo arquitectónico del menú iterativo (CLI), implementación central de los algoritmos de ordenamiento (Bubble, Selection, Insertion, Cocktail) y módulo de entrada/salida para el manejo de archivos CSV.
- **Alan Sánchez:** Implementación técnica de la mezcla aleatoria Fisher-Yates, desarrollo e integración del automatismo para el experimento de medición de tiempos computacionales y generación del motor de renderizado gráfico con Gnuplot.
- **Guliana Ruiz:** Implementación y optimización algorítmica de la búsqueda (Secuencial y Binaria por ID), desarrollo de la función comparadora universal multicriterio y estructuración analítica de la funcionalidad del ranking (Top N).

8. Anexos: Fragmentos de código relevantes

A continuación, se presentan los bloques de código más críticos desarrollados, los cuales demuestran la arquitectura de las soluciones aplicadas.

8.1. Comparador universal multi-criterio

Para cumplir el requerimiento de ordenar por cualquier campo sin duplicar el código de cada algoritmo, se utilizó un `switch-case` centralizado.

```
// Retorna > 0 si a > b, < 0 si a < b, y 0 si son iguales
int comparar(Deportista a, Deportista b, int criterio) {
    switch(criterio) {
        case 1: return a.id - b.id; // Orden por ID
        case 2: return strcmp(a.nombre, b.nombre); // Orden por Nombre
        case 3: return strcmp(a.equipo, b.equipo); // Orden por Equipo
        case 4:
            // Manejo estricto para valores flotantes (Puntaje)
            if (a.puntaje > b.puntaje) return 1;
            if (a.puntaje < b.puntaje) return -1;
            return 0;
        case 5: return a.competencias - b.competencias; //
    Competencias
        default: return 0;
    }
}
```

Listing 9: Función comparadora de estructuras.

8.2. Implementación optimizada: Bubble Sort

Muestra de la bandera de intercambio para forzar el escenario $\mathcal{O}(n)$ en el mejor caso.

```
void bubbleSort(Deportista *a, int n, int criterio) {
    int i, j, intercambio;
    for (i = 0; i < n - 1; i++) {
        intercambio = 0;
        for (j = 0; j < n - i - 1; j++) {
            if (comparar(a[j], a[j+1], criterio) > 0) {
                Deportista temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
                intercambio = 1; // Registro de movimiento
            }
        }
        if (intercambio == 0) break; // Optimizacion critica
    }
}
```

Listing 10: Bubble Sort con detección de arreglo ordenado.

8.3. Mezcla aleatoria Fisher-Yates

Esencial para garantizar el análisis del 'caso promedio' estadísticamente válido.

```
void mezclarDatos(Deportista *arreglo, int cantidad) {
    if (cantidad <= 1) return;
    for (int i = cantidad - 1; i > 0; i--) {
        int j = rand() % (i + 1); // Indice aleatorio
        Deportista temp = arreglo[i];
        arreglo[i] = arreglo[j];
        arreglo[j] = temp;
    }
}
```

Listing 11: Algoritmo Fisher-Yates.